

FLASK FRAMEWORK

Professional Developer Manual (Beginner → Advanced → Production)

Architecture • Code • Case Studies • Deployment • Roadmap

1. Introduction to Flask

Flask is a micro web framework written in Python. It is designed to be lightweight, extensible, and easy to learn.

Unlike monolithic frameworks, Flask provides only routing, request handling, and templating out of the box.

This design makes Flask ideal for APIs, microservices, and scalable backend systems.

Use Cases

- REST APIs - Microservices - SaaS backends - Machine learning model serving

2. Flask Architecture

Flask is built on the Web Server Gateway Interface (WSGI) standard.

Werkzeug handles HTTP mechanics, while Jinja2 handles templating.

Request Lifecycle

Client → WSGI Server → Flask App → View Function → Response

3. Environment Setup

```
python -m venv venv
source venv/bin/activate
pip install flask
```

Project Layout

```
app/
■■■ __init__.py
■■■ routes.py
■■■ models.py
■■■ templates/
■■■ static/
run.py
```

4. Routing and Views

Routes bind URLs to Python functions.

```
@app.route('/')  
def home():  
    return 'Home Page'
```

Dynamic Routing

```
@app.route('/user/<int:id>')  
def user(id):  
    return f'User ID: {id}'
```

5. Templates and Jinja2

Jinja2 enables dynamic HTML rendering with template inheritance.

```
{% extends "base.html" %}
{% block content %}
<h1>{{ title }}</h1>
{% endblock %}
```

6. Database Integration

Flask integrates with relational databases via SQLAlchemy.

```
class Product(db.Model):  
    id = db.Column(db.Integer, primary_key=True)  
    name = db.Column(db.String(100))
```

7. Authentication and Security

Security is a critical aspect of production Flask applications.

- Password hashing - JWT authentication - CSRF protection - Secure cookies

8. REST API Development

Flask is widely used for RESTful services.

```
@app.route('/api/items', methods=['GET'])
def items():
    return jsonify(items)
```

9. Case Study: Blog Platform

A real-world blog application includes authentication, authorization, and content management.

Flask Blueprints help separate concerns.

10. Case Study: ML Model API

Flask is commonly used to deploy ML models as HTTP services.

```
@app.route('/predict', methods=['POST'])
def predict():
    data = request.json
    return jsonify(model.predict(data).tolist())
```

11. Deployment

Production deployment uses Gunicorn behind Nginx.

Docker ensures environment consistency.

12. Performance and Scaling

- Caching with Redis - Horizontal scaling - Background tasks

13. Testing

Testing ensures application reliability.

```
def test_home(client):  
    assert client.get('/').status_code == 200
```

14. Learning Roadmap

Beginner → Intermediate → Advanced progression:

Flask basics → Templates → Databases → APIs → Security → Deployment

15. Best Practices

Keep routes thin, logic in services.

Never use Flask dev server in production.

16. Conclusion

Flask remains one of the most flexible Python web frameworks for professional development.